SEARCH PROBLEMS & BEST-FIRST SEARCH

How do you formalize "finding a solution" precisely enough for a machine to do it?

You want to drive to a city that you have never visited (not using a map app; but from scratch). What do you actually need to know to solve this problem?

- Where you are now
- Where you want to end up
- What moves are available at each point
- What each move costs
- How you would recognize you have arrived

Defining a Search Problem

State Space	S	The set of all possible states
Initial State	$s_0 \in S$	The state where the agent starts
Goal States	$\subseteq S$	The state(s) that count as a solution
Actions(s)	$S \rightarrow$ *Set of subsets of* A *A: Set of all actions*	Actions available in state s
Result(s, a)	$S \times A \rightarrow S$	The transition model - the state reached after action a in s
Action-Cost(s, a, s′)	$S \times A \times S \rightarrow \mathbb{R}$	Numeric cost of taking action a in s, arriving at s′

Route-Finding Problem

State Space	Cities on the map
Initial State	Starting city
Goal States	Destination city/cities
Actions(s)	Drive to any city directly connected to s
Result(s, a)	City reached by taking a from city s
Action-Cost(s, a, s′)	Road distance between s and s′ (km)

8-Puzzle

State Space	The set of all arrangements of the 8 numbered tiles (1–8) and one blank on a 3×3 grid: 9!/2 = 181,440 reachable configurations
Initial State	The configuration of tiles the puzzle starts in
Goal States	The single arrangement with tiles in order (typically 1–8 reading left-to-right, top-to-bottom, blank in the bottom-right)
Actions(s)	Move the blank Up, Down, Left, or Right; the applicable subset depends on where the blank currently is
Result(s, a)	The arrangement obtained by sliding the tile adjacent to the blank into the blank's position (equivalently, swapping the blank with that neighboring tile)
Action-Cost(s, a, s′)	1 for every move - each slide costs the same regardless of which tile moves

Vacuum Cleaner World

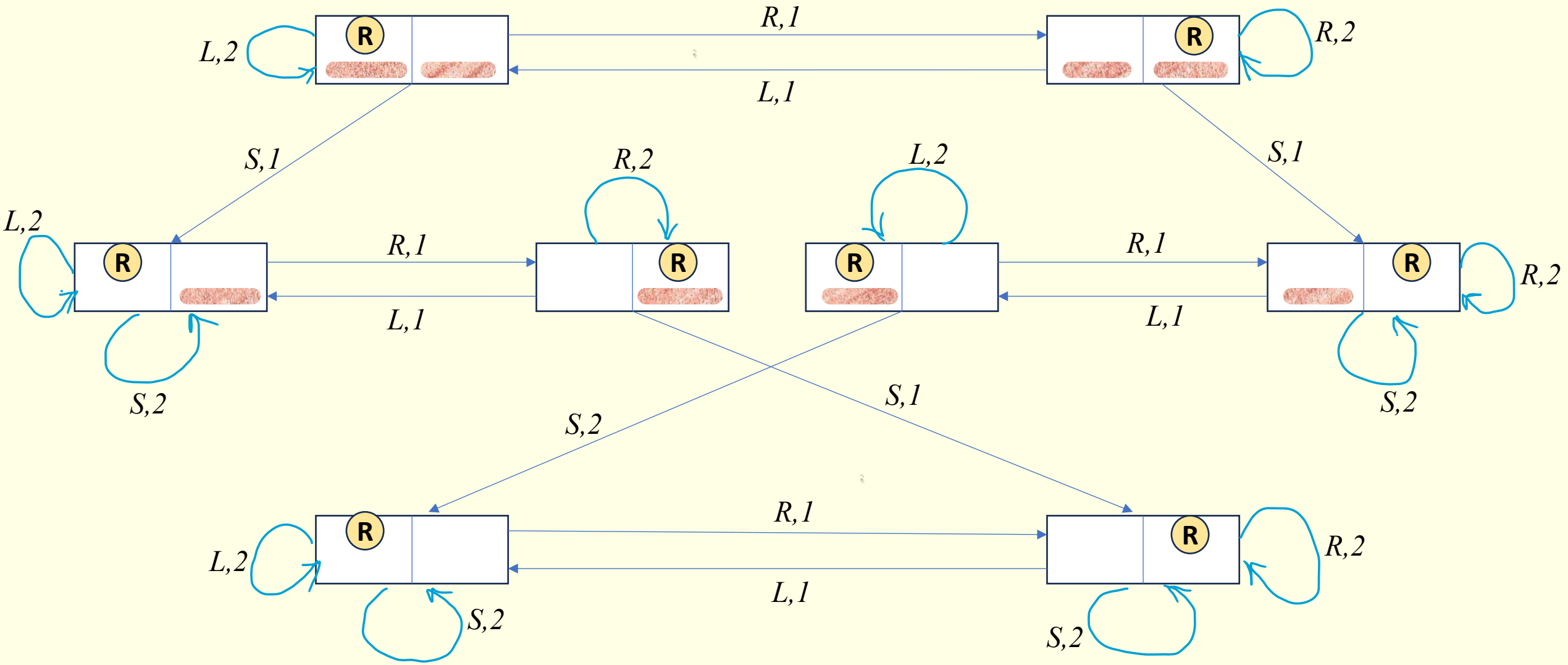
Suppose the vacuum cleaner world had n rooms, where each room can be either *Clean* or *Dirty*, and the robot can be in any one of the n rooms.

State Space	The set of all possible $n.2^n$ states
Initial State	The state in the beginning
Goal States	The set of all n states in the state space in which all rooms are *Clean* (The robot can be in any of the n rooms.)
Actions(s)	*Suck* (S), *MoveLeft* (L), *MoveRight* (R)
Result(s, a)	The state resulting from applying action a at state s
Action-Cost(s, a, s′)	1 for every move (A better Action-Cost function can be to keep a slightly higher cost when the action leads nowhere, that is, for all (s, a, s′) with s = s′. This will help to avoid the robot making unnecessary moves.)

The **State Space Graph** for a search problem is an edge-weighted, directed graph having nodes as the states in the search problem, edges as the actions leading from one state (node) to another, and the edge weights as the corresponding action costs.

If the state space is undirected, each edge is considered to be bidirectional.

State-Space Graph for a vacuum cleaner world with two rooms



Note: Each pair a, c denotes action a with cost c.

Best-First Search: The General Algorithm

Maintain a frontier of nodes not yet expanded, ordered by an evaluation function $f(n)$. Repeatedly remove the "best" node; if it is a goal, stop; otherwise, expand it.

1. Frontier ← priority queue with only the initial state, ordered by $f(n)$.
2. Reached ← table recording the cheapest known path found so far to each state.
3. If Frontier is empty, return failure.
4. Remove the node with lowest $f(n)$ from Frontier.
5. If that node is a goal state, return the solution.
6. For each child:
   if unseen OR reached with lesser value of $f(n)$ than before:
   update Reached and Frontier.
7. Repeat from Step 3.

State Space vs. Search Tree

State Space / State Space Graph

A fixed mathematical object - the directed graph of all states and the actions connecting them. It does not change as the algorithm runs.

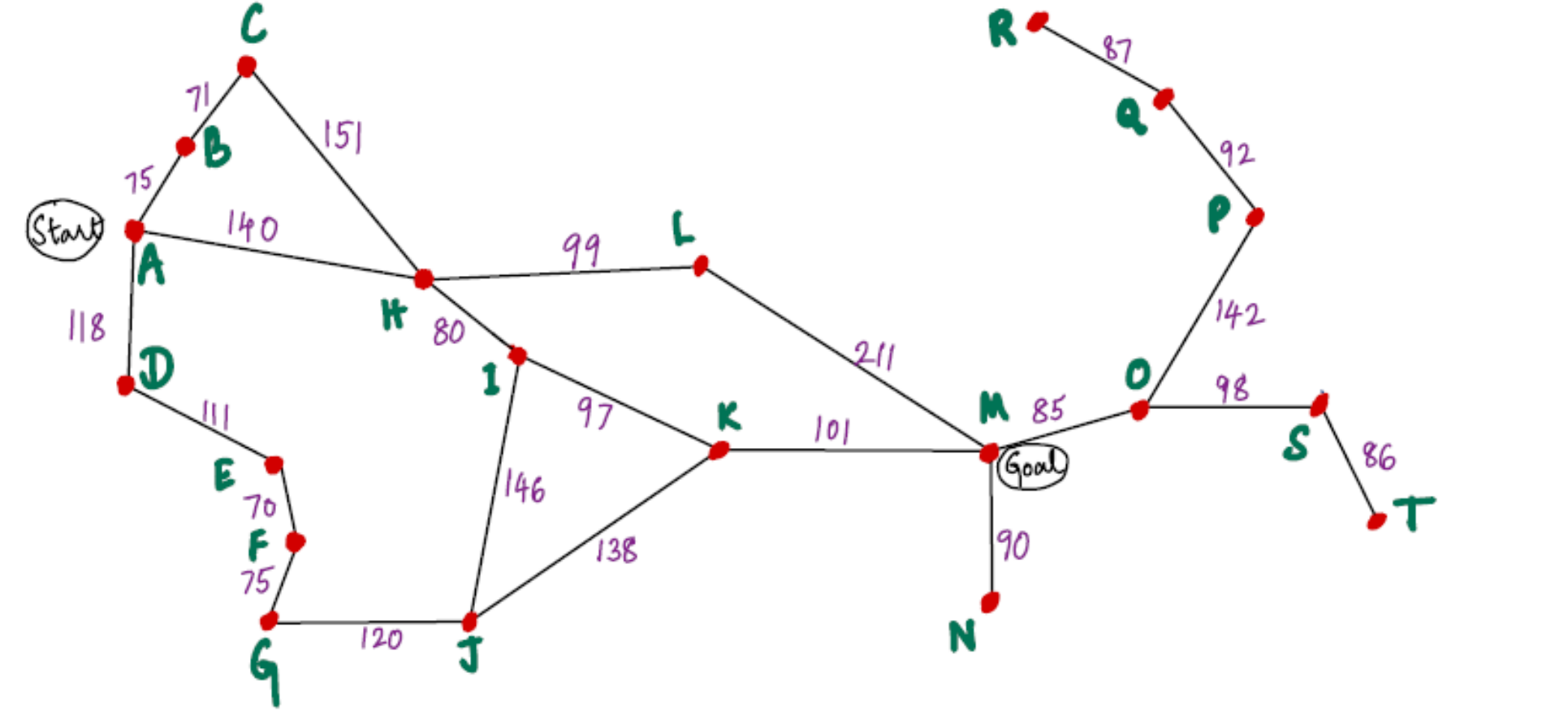
Search Tree

What the algorithm builds while exploring. The same state can appear more than once, reached via different paths.

Different choices of $f(n)$ in best-first search correspond to different algorithms:

$f(n)$	Algorithm
Path cost $g(n)$	**Uniform-Cost Search (UCS)**
$depth(n)$	**Breadth-First Search (BFS)** (special case of UCS)
Negative of $depth(n)$ (without "reached with lesser value of $f(n)$ than before" condition in Step 6)	**Depth-First Search (DFS)**

The following is the state space graph corresponding to a route-finding problem where the nodes correspond to some of the cities in Romania and the edge weights (costs) are the distances among the corresponding cities.



Tracing Uniform Cost Search algorithm for the Romania route-finding problem:

- $f(n) = g(n)$, the path cost from the initial state to state n.

Initialization : frontier $\leftarrow \{A_0\}$

reached $\leftarrow \{A_0\}$

While loop :

(1) node $\leftarrow$ POP (frontier) $= A$.

EXPAND (A) :

Child node n	B	D	H
Parent	A	A	A
$g(n)$	75	118	140

frontier = { B, D, H }
75 118 140

reached = { A, B, D, H }
0 75 118 140

② node ← POP(frontier) = B.

EXPAND (B):

Child node n	A	C
Parent	B	B
g(n)	150	146

frontier = { D, H, C }
118 140 146

reached = { A, B, D, H, C }
0 75 118 140 146

③ node ← POP(frontier) = D.

EXPAND(D):

Child node n	A	E
Parent	D	D
g(n)	236	229

frontier = { H, C, E } (H: 140, C: 146, E: 229)

reached = { A, B, D, H, C, E } (A: 0, B: 75, D: 118, H: 140, C: 146, E: 229)

④ node ← POP(frontier) = H.

EXPAND(H):

Child node n	A	C	I	L
Parent	H	H	H	H
g(n)	280	291	220	239

frontier = {C, E, I, L} (146, 229, 220, 239)

reached = {A, B, D, H, C, E, I, L} (0, 75, 118, 140, 146, 229, 220, 239)

(5) node ← POP(frontier) = C.

EXPAND(C):

Child node n	B	H
Parent	C	C
g(n)	217	297

frontier = {E, I, L} (229, 220, 239)

reached = {A, B, D, H, C, E, I, L} (0, 75, 118, 140, 146, 229, 220, 239)

(6) node ← POP(frontier) = I.

EXPAND(I):

Child node n	H	J	K
Parent	I	I	I
g(n)	300	366	317

frontier = {E, L, J, K}
229 239 366 317

reached = {A, B, D, H, C, E, I, L, J, K}
0 75 118 140 146 229 220 239 366 317

(7) node ← POP(frontier) = E.

EXPAND(E):

Child node n	D	F
Parent	E	E
g(n)	340	299

frontier = {L, J, K, F}
239 366 317 299

reached = {A, B, D, H, C, E, I, L, J, K, F}
0 75 118 140 146 229 220 239 366 317 299.

(8) node ← POP(frontier) = L.

EXPAND(L):

Child node n	H	M
Parent	L	L
g(n)	338	450

frontier = {J, K, F, M}
366 317 299 450

reached = {A, B, D, H, C, E, I, L,
0 75 118 140 146 229 220 239
J, K, F, M}.
366 317 299 450

(9) node ← POP(frontier) = F.

EXPAND(F):

Child node n	E	G
Parent	F	F
g(n)	369	374

frontier = {J, K, M, G}
366 317 450 374

reached = {A, B, D, H, C, E, I, L,
0 75 118 140 146 229 220 239
J, K, F, M, G}.
366 317 299 450 374

(10) node ← POP(frontier) = K.

EXPAND(K):

Child node n	I	J	M
Parent	K	K	K
g(n)	414	455	418

frontier = { J, ~~M~~ 418, G }
366 374

reached = { A, B, D, H, C, E, I, L, J, K, F, ~~M~~ 418, G }.
0 75 118 140 146 229 220 239 366 317 299 374

(11) node ← POP(frontier) = J.

EXPAND(J):

Child node n	G	I	K
Parent	J	J	J
g(n)	486	512	504

frontier = { M, G }
418 374

reached = { A, B, D, H, C, E, I, L,
0 75 118 140 146 229 220 239
J, K, F, M, G }.
366 317 299 418 374

(12) node $\leftarrow$ POP(frontier) = G.

EXPAND(G):

Child node n	F	J
Parent	G	G
g(n)	449	494

frontier = { M } (418)

reached = { A (0), B (75), D (118), H (140), C (146), E (229), I (220), L (239), J (366), K (317), F (299), M (418), G (374) }.

(13) node $\leftarrow$ POP(frontier) = M.

IS-GOAL(M) = true.

return M.

The optimal cost path from A to M is

$$A \xrightarrow[140]{} H \xrightarrow[80]{} I \xrightarrow[97]{} K \xrightarrow[101]{} M.$$

The total path cost is $140 + 80 + 97 + 101 = 418$.

Some modifications to improve efficiency when $f(n) = \pm \text{depth}(n)$:

When $f(n) = depth(n)$ (BFS case) :

- A FIFO queue can be used for frontier instead of a priority queue.
- Goal test can be done as soon as a node a generated; because no node will be visited a second time at a smaller depth.
- "reached with lesser value of $f(n)$ than before" condition in Step 6 can be removed; because it will never be true.

When $f(n) = -depth(n)$ (DFS case) :

- A LIFO queue can be used for frontier instead of a priority queue.
- Goal test can be done as soon as a node a generated; because DFS anyway does not guarantee an optimal path (even if goal test is done after popping the node).

From Uninformed (Blind) Search to Informed Search

Uninformed search (BFS, DFS, UCS) explores blindly — it has no sense of which unexplored node is "closer" to the goal.

As the state space grows, blind exploration becomes too slow to be practical.

Idea: What if the search could *estimate* how promising a state is, before expanding it?

Heuristic functions

- A heuristic function $h(n)$ is an estimate of the cost from node n to the nearest goal.
- We have $h(\text{goal}) = 0$.
- The heuristic function is problem-specific. It is built using domain knowledge.

Examples:

- Route-finding problems: $h(n) =$ Straight line distance from city n to the goal (destination).
- 8-Puzzle: $h(n) =$ Number of misplaced tiles in state n.

Different choices of $f(n)$ in best-first search correspond to different informed search algorithms:

$f(n)$	Algorithm
Heuristic function $h(n)$	**Greedy Best-First Search**
Sum of path cost and heuristic $g(n) + h(n)$	**A* Search**

Consider the following heuristic for the Romania route finding problem. Here $h(n)$ is the straight-line distance from city n to the goal city (M) for each n.

$h(A) = 366$
$h(B) = 374$
$h(C) = 380$
$h(D) = 329$
$h(E) = 244$
$h(F) = 241$
$h(G) = 242$

$h(H) = 253$
$h(I) = 193$
$h(J) = 160$
$h(K) = 100$
$h(L) = 176$
$h(M) = 0$
$h(N) = 77$

$h(O) = 80$
$h(P) = 199$
$h(Q) = 226$
$h(R) = 234$
$h(S) = 151$
$h(T) = 161$

Implementation of Greedy Best-first Search algorithm for the Romania route-finding problem:

- $f(n) = h(n)$, the heuristic function.

Note: As $h(n)$ has a fixed value for every node n, it is not required to check the condition whether a state is reached with a better value of $f(n) = h(n)$. Hence, it is not required to keep track of $f(n)$ values in reached list in the case of greedy best-first-search.

Initialization : frontier $\leftarrow \{A\}_{366}$

reached $\leftarrow \{A\}_{366}$

While loop :

① node $\leftarrow$ POP (frontier) = A.

EXPAND (A) :

Child node	B	D	H
Parent	A	A	A

frontier $= \{B, D, H\}_{374\ 329\ 253}$

reached $= \{A, B, D, H\}$.

② node ← POP(frontier) = H.

EXPAND (H):

Child node	A	C	I	L
Parent	H	H	H	H

frontier = {B, D, C, I, L}
374 329 380 193 176

reached = {A, B, D, H, C, I, L}

③ node ← POP(frontier) = L.

EXPAND (H):

Child node	H	M
Parent	L	L

frontier = {B, D, C, I, M}
374 329 380 193 0

reached = {A, B, D, H, C, I, L, M}

(4) node ← POP(frontier) = M.

IS-GOAL(M) = true.

return M.

The path from A to M is $A \xrightarrow[140]{} H \xrightarrow[99]{} L \xrightarrow[211]{} M$.

The total path cost is $140 + 99 + 211 = 450$.

⊛ Note that this is not the cost-optimal path.

Tracing A* Search algorithm for the Romania route-finding problem:

- $f(n) = h(n) + g(n)$, the sum of heuristic function and path cost.

Initialization : frontier $\leftarrow \{A_0\}$

reached $\leftarrow \{A_0\}$

While loop :

① node $\leftarrow$ POP (frontier) $= A$.

EXPAND (A) :

Child node n	B	D	H
Parent	A	A	A
$g(n) + h(n)$	449	447	393

frontier = { B, D, H } (449, 447, 393) reached = { A, B, D, H } (366, 449, 447, 393).

② node ← POP(frontier) = H.

EXPAND (H):

Child node n	A	C	I	L
Parent	H	H	H	H
g(n) + h(n)	646	671	413	415

frontier = { B, D, C, I, L } (449, 447, 671, 413, 415) reached = { A, B, D, H, C, I, L } (366, 449, 447, 393, 671, 413, 415)

(3) node ← POP(frontier) = I.

EXPAND (I):

Child node n	H	J	K
Parent	I	I	I
g(n) + h(n)	553	526	417

frontier = { B, D, C, L, J, K }
449 447 671 415 526 417

reached = { A, B, D, H, C, I, L, J, K }
366 449 447 393 671 413 415 526 417

(4) node ← POP(frontier) = L.

EXPAND (L):

Child node n	H	M
Parent	L	L
g(n) + h(n)	591	450

frontier = { B, D, C, J, K, M }
449 447 671 526 417 450

reached = { A, B, D, H, C, I, L, J, K, M }.
366 449 447 393 671 413 415 526 417 450

(5) node ← POP(frontier) = K.

EXPAND(K):

Child node n	I	J	M
Parent	K	K	K
g(n) + h(n)	607	615	418

frontier = { B, D, C, J, K, ~~M~~ }
449 447 671 526 417 ~~450~~ 418

reached = { A, B, D, H, C, I, L, J, K, ~~M~~ }.
366 449 447 393 671 413 415 526 417 ~~450~~ 418

⑥ node ← POP(frontier) = M.

IS-GOAL(M) = true.

return M.

The path from A to M is $A \xrightarrow[140]{} H \xrightarrow[80]{} I \xrightarrow[97]{} K \xrightarrow[101]{} M$.

The total path cost is $140 + 80 + 97 + 101 = 418$.

This is optimal.

Admissible Heuristic

A heuristic h is said to be admissible if for every node n,

$$h(n) \leq p(n)$$

where $p(n)$ is the optimal path cost from node n to goal.

This means that a heuristic is admissible if it never overestimates the cost to reach a goal.

Result:
If heuristic h is admissible, then A^* algorithm returns an optimal solution.

Proof: Suppose C^* is the optimal cost from the initial node to the goal node. Let

$$\text{Initial} = n_0 \to n_1 \to n_2 \to \cdots \to n_k = \text{Goal}$$

be an optimal solution (path). We prove the result by contradiction.

Suppose A^* algorithm returns a non-optimal path

$$\text{Initial} = n_0 \to n_1{}' \to n_2{}' \to \cdots \to n_l{}' = \text{Goal}$$

with cost $C > C^*$. Then there must exist i $(1 \leq i \leq k-1)$ such that node n_i is generated but not expanded by A^*.

For this n_i, $f(n_i) > C$; because node $n_l{}'$ got prioritized and popped from the frontier with $f(n_l') = C$. As $C > C^*$, we have $f(n_i) > C^*$.

Proof (continued):

Now

$$
\begin{aligned}
f(n_i) &= g(n_i) + h(n_i) && \text{(by definition)} \\
&= g^*(n_i) + h(n_i) && \text{(because } n_i \text{ is on the optimal path)} \\
&\leq g^*(n_i) + p(n_i) && \text{(because } h \text{ is admissible)} \\
&= C^*.
\end{aligned}
$$

Therefore, $f(n_i) \leq C^*$, a contradiction to $f(n_i) > C^*$.

Consistent Heuristic

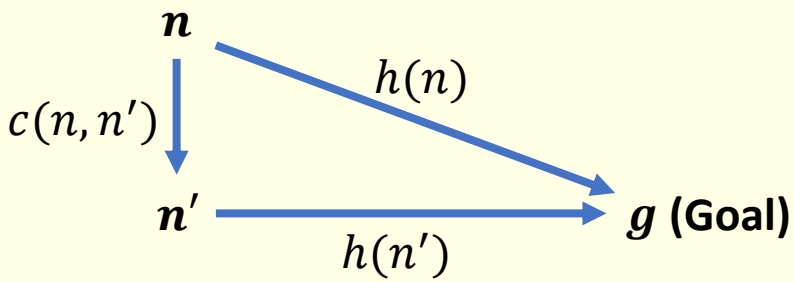
A heuristic h is said to be consistent if for every node n and for every successor n' of n generated by an action a,

$$h(n) \leq c(n, a, n') + h(n').$$

For simplicity, we can write

$$h(n) \leq c(n, n') + h(n').$$

Note that this is a form of triangular inequality:



Result:
Let h be a heuristic. If h is consistent, then h is admissible.

Proof: Suppose h is consistent. We need to show that $h(n) \leq p(n)$ for each node n.
Suppose n is a node from which there is no path to the goal g. Then $p(n) = \infty$, and hence $h(n) \leq p(n)$ holds trivially.
The rest of the proof is by mathematical induction on the number of intermediate nodes k in a cost-optimal path from the given node to the goal g.

Base case: Suppose n is a node with $k = 0$, that is, the goal g is a successor of n.
By consistency, $h(n) \leq c(n, g) + h(g)$.
As $c(n, g) = p(n)$ and $h(g) = 0$, we get $h(n) \leq p(n)$.

Proof (continued):

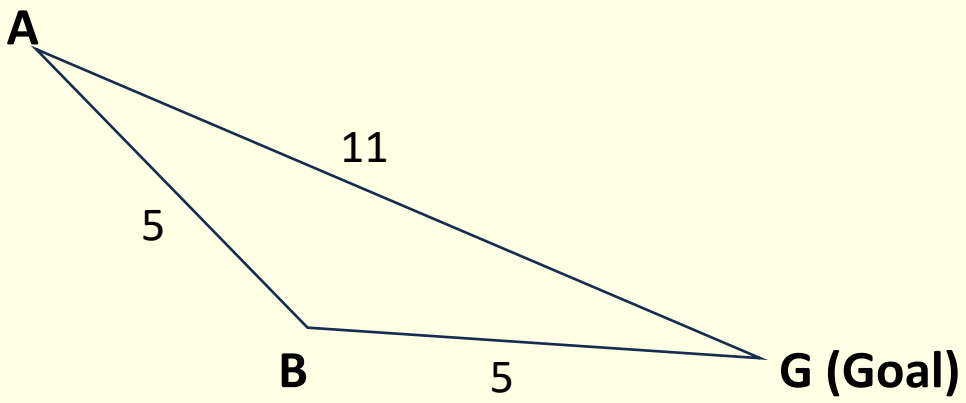
Induction hypothesis: Suppose $h(n) \leq p(n)$ for any node n with $k-1$ intermediate nodes on a cost-optimal path from n to the goal g.

Induction step: Suppose n is a node such that there are k intermediate nodes on a cost-optimal path from n to g. Let $n \to a_1 \to a_2 \to \cdots \to a_k \to g$ be a cost-optimal path.
By consistency, we have $h(n) \leq c(n, a_1) + h(a_1)$.
Using induction hypothesis on node a_1, we get $h(a_1) \leq p(a_1)$.
Hence, $h(n) \leq c(n, a_1) + p(a_1) = p(n)$, that is, $h(n) \leq p(n)$.

Therefore, $h(n) \leq p(n)$ for all nodes n by induction.
Hence h is admissible.

Note: An admissible heuristic need not be consistent.

An example of such a state space graph and a heuristic is given below:



Let h be as follows:
$h(A) = 9$
$h(B) = 3$
$h(G) = 0$

Note that h is admissible; because
$h(A) \leq p(A) = 10$
$h(B) \leq p(B) = 5$
$h(G) \leq p(G) = 0.$

But h is not consistent because
$h(A) = 9 > c(A, B) + h(B) = 5 + 3 = 8.$

We have proved the following:

- If heuristic h is admissible, then A^* algorithm returns an optimal solution.
- If h is consistent, then h is admissible.

Therefore, the following is also true:

If heuristic h is consistent, then A^* algorithm returns an optimal solution.

Some examples of commonly used heuristics

- **Route-finding problems:**

  $h(n) =$ Straight line distance from city n to the goal (destination).

  $h(n)$ is admissible and consistent.

- **8-Puzzle:**

  $h(n) =$ Number of misplaced tiles in state n.

  $h(n)$ is admissible, and consistent if all action costs are taken to be 1.

- **8-Puzzle:**

  $h(n) =$ Sum of Manhattan distances of the misplaced tiles to their respective correct positions.

  $h(n)$ is admissible, and consistent if all action costs are taken to be 1.

- ***N* - Queens Problem** (The problem of placing N queens on an $N \times N$ chessboard such that no two queens attack each other):
  $h(n) =$ Number of (directly or indirectly) attacking pairs of queens.
  $h(n)$ is not admissible, and hence not consistent.